1. Integration File Architecture

The integration logic is divided across the following backend and frontend layers:

Backend (Java / Spring Boot)

- **IndiaMART Sync Service**: IndiamartIntegrationService.java - Contains the scheduling, API connection, JSON parsing, de-duplication, and database persistence logic.
- **TradeIndia (Tradelink) Sync Service**: TradeIndiaIntegrationService.java - Exposes a similar structure for pulling directory inquiries from TradeIndia.
- **REST Configuration Controller**: IntegrationConfigController.java - Defines endpoints /api/integrations/... to fetch configs, update API keys, manually trigger a sync, and run connection checks.
- **Database Entity**: IntegrationConfig.java - Maps parameters stored in the crm_integration_config table (keys, sync status, auto-assignee mapping, number of pulled leads, etc.).

Frontend (React / Tailwind)

- **Dashboard Page**: IntegrationsPage.jsx - Provides a visual panel for Super Admins/Company Admins to input credentials, toggle status, and manually test/sync.

---

2. How Data is Pulled from IndiaMART

The integration pulls leads through the following sequential workflow:

A. Execution Trigger (Scheduler & Manual)

- **Automatic Scheduled Polling**: Inside IndiamartIntegrationService.java:L29-L48, a background scheduler runs every **5 minutes and 15 seconds** using:

java

@Scheduled(fixedRate = 315000)

public void fetchIndiamartLeads()

- **Manual Sync**: Users can trigger an immediate pull by clicking **Sync Now** in the UI, which routes to IntegrationConfigController.java:L209-L251.

B. API Rate Limiting Guard

IndiaMART rejects calls occurring within 5 minutes of each other. The service protects against this in two ways:

- **Time Check**: Checks if config.getLastSyncTime() was within the last 290 seconds. If so, it skips execution to avoid being blocked.
- **Error Handling**: If a rate limit block or key error is encountered, the exception is caught, syncStatus is set to "FAILURE", and the sync time is set to LocalDateTime.now() to prevent infinite loops.

C. API Request Construction

- The service builds a query URL dynamically:

[API_URL]?glusr_crm_key=[API_KEY]&start_time=[START_TIME]&end_time=[END_TIME]

- **Time Windows**:
    - end_time is the current local time.
    - start_time goes back to **5 minutes before the last successful sync** (to prevent missing overlapping boundary leads), or defaults to the last 7 days (IndiaMART's maximum single call limit) if running for the first time.
    - Formatted as dd-MMM-yyyyHH:mm:ss (e.g. 10-Jun-202612:52:00).

D. Data Extraction & De-duplication

- A GET request is executed using Spring RestTemplate.
- The service parses the JSON response using ObjectMapper:
    - **Code 200 (Leads Available)**: Iterates over the RESPONSE JSON array.
    - **Code 204 (No Leads)**: Sets the sync status to SUCCESS and updates the last sync timestamp.
    - **Other Codes**: Throws a RuntimeException with IndiaMART's error message.
- **De-duplication check**: For every entry, it reads the UNIQUE_QUERY_ID and checks the database repository:

java

if (!leadRepository.existsByUniqueQueryId(queryId)) { ... }

This ensures that inquiries are never saved multiple times.

E. Lead Creation & Assignment

- If the lead is unique, a new Lead object is built:
    - **Sender Info**:
      Maps SENDER_NAME, SENDER_MOBILE, SENDER_EMAIL, SENDER_COMPANY, SE
      NDER_CITY, etc.
    - **Message Content**: Maps QUERY_MESSAGE to leadReason.
    - **Source**: Flagged as "India MART".
    - **Auto-Assignment**: Assigned to the team member specified
      in autoAssignUserId configuration.
- The lead is persisted using leadRepository.save().
- Finally, the configuration status, synchronization timestamp, and total lead count are updated.

NOTE

All your background tasks (dev servers for frontend and backend) were shut down due to a system container restart. Let me know if you would like me to restart them for you to test these sync features!